OrchestraBench: Evaluating Multi-Agent Orchestration Failure Modes, Recovery, and Decomposition Quality

Anonymous Author(s)

ABSTRACT

Multi-agent orchestration frameworks (AutoGen, LangGraph, CrewAI, Anthropic Agents SDK) are moving from demos to production, yet they can only be compared on *task accuracy* — not on *reliability*. Existing agent benchmarks measure whether a pipeline succeeds, but cannot diagnose *why* it failed, *where* a cascade began, or *which* routing decision caused the breakdown. We present **OrchestraBench**, a benchmark that evaluates how multi-agent systems **fail, recover, and decompose** as first-class, comparable metrics. OrchestraBench contributes (i) a controlled, seed-reproducible **failure-injection harness** over templated enterprise workflows, (ii) **cascade-radius** and per-failure-mode recovery as primary metrics, and (iii) a **routing-policy comparison** with bootstrap confidence intervals and paired tests where applicable. Our first experiment isolates the routing-mechanism question: on a 26-case gold-labelled diagnostic, a keyword/flag heuristic — representative of production rule-based routers — scores **0% on adversarial cases** (misleading or missing surface flags), while a model-driven router that reasons over intent scores **100%**, matching the oracle on this diagnostic. This indicates that the heuristic's blind spot is a property of the routing *mechanism*, not the diagnostic's inherent difficulty. Two further experiments, run with a real Claude agent over a **verifiable arithmetic dependency chain** as a controlled mechanism probe, quantify a reliability axis existing benchmarks leave undermeasured: across five MAST failure modes, the four latent/semantic modes fail completely (final-task success **0.0**) while a tool fault is fully recovered (**1.0**), and **retry does not repair the latent modes** — it extends a corrupted trace rather than containing it; and cascade radius **scales monotonically with pipeline depth** (mean 1.0 / 2.9 / 5.0 at depths 3 / 5 / 7) — the stages-traversed signature MAS-FIRE leaves unquantified. A policy-conditioned extension further suggests that containment can improve under a stronger self-correction semantics: with trusted upstream state, an LLM router cuts latent cascade radius ~5.5× over rule-based baselines at depth 7 — but an ablation shows this gain is largely the trusted-state signal, not autonomous detection. We frame these as controlled-chain mechanism probes, not domain-workload claims (§7).

CCS CONCEPTS

• **Computing methodologies → Multi-agent systems**; • **General and reference → Evaluation**; • **Software and its engineering →** *Software reliability*.

KEYWORDS

LLM orchestration, multi-agent systems, benchmarking, failure recovery, cascade propagation

1 INTRODUCTION

Multi-agent systems increasingly orchestrate several specialized agents — routers, retrievers, tool-callers, planners — behind a single task. When such a pipeline produces a wrong or low-quality answer in production, the failure is often *silent*: a misrouted task still yields a plausible-looking response. Current benchmarks (AgentBench [7], OdysseyBench [9], SWE-Bench [5]) report final task success and so cannot tell teams **which** orchestration decision failed, **how far** an error propagated, or **whether** intelligent routing was worth its added complexity. This blocks the most basic production decision: choosing and hardening an orchestration framework on *reliability* rather than ergonomics.

Contributions.

(1) **OrchestraBench**, a benchmark that treats orchestration *failure handling* as the unit of evaluation: controlled failure injection, per-failure-mode recovery, and cascade propagation.
(2) A **seed-reproducible** workflow + failure-injection harness (scenarios regenerable from seeds; aggregates recomputable from committed artifacts) with templated enterprise workflow families.
(3) A **routing-policy comparison** (Fixed / Heuristic / LLM / Oracle) with bootstrap confidence intervals and paired significance testing where applicable, isolating *when* intelligent routing beats a zero-decision baseline.

Research questions.

- **RQ1 (routing value)**: When does intelligent routing (heuristic or LLM) actually beat a zero-decision baseline?
- **RQ2 (mechanism)**: Is routing reliability a property of task *difficulty*, or of the routing *mechanism* (keyword/flag matching vs. reasoning over intent)? *(Answered by Exp 1.)*
- **RQ3 (attribution)**: Can a pipeline failure be attributed to a specific failure mode *and* stage, reproducibly across runs?
- **RQ4 (cascade)**: How far does a single seeded error propagate downstream, and which routing/recovery strategies contain it?

2 RELATED WORK

OrchestraBench sits among recent work that *observes*, *attributes*, or *proposes* fixes for multi-agent failure; we differ by making controlled injection, cascade radius, and routing-policy mechanism the unit of analysis (Table 1).

Motivating data point. Beyond the Strongest [8] reports that on GPQA-Diamond at least one agent was correct in **95.5%** of cases, yet orchestration reached only **87.4%** — i.e. orchestration *discards* ~8 points of individually-recoverable correctness. OrchestraBench is built to attribute exactly this loss.

Table 1: OrchestraBench versus closely related work.

Prior work	What it does	How OrchestraBench differs
MAST [2] (14 failure modes, 1,600+ traces, κ=0.88)	Empirically-grounded *taxonomy* of observed MAS failures	We *inject* that taxonomy under seed-controlled conditions and measure recovery + cascade *per mode* — MAST observes, we intervene
MAS-FIRE [4]	Fault injection + reliability evaluation; intra-/inter-agent fault taxonomy	**Closest prior work** (§2.1). We differ on emphasis: cascade *radius across pipeline depths* + *routing-policy comparison* as the unit of analysis, not reliability scoring alone
TraceElephant [3] (220 annotated traces)	Benchmark for post-hoc failure attribution in MAS	Attribution is observational; we add controlled seeding + cascade radius as *primary* metrics
Agents Failure Attribution [12] (Who&When; 127 MAS)	Post-hoc attribution to the responsible agent/step	Same: attribution is a means here, not the product
Orchestration-pattern benchmark [6] (10k SEC filings)	Compares sequential / parallel / hierarchical / reflexive architectures on a cost-accuracy Pareto	Architecture comparison on accuracy/cost; we evaluate *failure handling* and routing *mechanism* — complementary
AdaptOrch [11]	Task-adaptive orchestration *framework* (+12–23%)	A method; OrchestraBench is the benchmark that would evaluate it
From Spark to Fire [10]	Errors cascade exponentially in pipelines	We operationalize this into a measurable cascade-radius benchmark

The gap. Existing work either *observes* failures (MAST [2]), *attributes* them post-hoc (TraceElephant [3], [12]), or *proposes* better orchestration (AdaptOrch [11]). The one work that also *injects* (MAS-FIRE [4]) scores reliability but does not make cascade radius or routing-policy mechanism its unit of analysis. OrchestraBench's contribution is a **controlled, reproducible injection harness that measures recovery and cascade containment as first-class metrics, compared across routing policies.**

2.1 Positioning Relative to Closest Prior Work

An honest audit (the space moved fast in early 2026) surfaces one direct overlap and our response:

- **MAS-FIRE [4] already performs controlled fault injection + reliability evaluation**, overlapping our Exp 2/3 substantially. Our differentiation is *not* "we inject failures" (they do too); it is (a) **cascade radius across pipeline depths** as a first-class comparable metric, (b) failure handling measured **per routing policy**, and (c) the **routing-mechanism result** of Exp 1, which MAS-FIRE does not address. In our reading, MAS-FIRE does not quantify cascade depth as a stages-traversed metric and does not vary routing policy; its axes are topology (MetaGPT / Table-Critic / CAMEL), fault-type (15), and model. OrchestraBench therefore complements MAS-FIRE by making cascade depth and routing policy first-class evaluation axes.
- **Attribution benchmarks [3, 12] are observational/post-hoc** — a different instrument from seeded, reproducible injection + recovery.
- **Internal overlap: IntentBench [1]** evaluates whether the agent's *understanding of intent* is correct (intent-spec vs. syntactic tool-call correctness); OrchestraBench evaluates which *orchestration action* to take given a correct understanding and how those decisions cascade. We measure the **decision policy and its failure propagation**, not intent correctness — complementary ends of the pipeline, no measurement overlap.

3 THE ORCHESTRABENCH BENCHMARK

Workflow suites. Synthetically generated from templated task graphs (stages, inter-task dependencies, per-task `complexity_score`), screened by two annotators for realism. Three enterprise families: finance approval (4 stages), HR onboarding (5 stages), DevOps deployment (5 stages).

Routing decision space. Each task is routed to one of four actions: `DIRECT_TOOL`, `CODE_EXECUTION`, `DECOMPOSE`, `REASON_ONLY`.

Gold labels. Routing and decomposition labels are produced independently by two annotators with adjudication on disagreement.

Failure injection. Failure scenarios are seeded **deterministically** on top of the suites via the `FailureInjector`, so each scenario is reproducible from a seed.

Metrics. Primary reported metrics are routing accuracy, per-failure-mode recovery rate, **cascade radius**, time-to-detection, recovery completeness, and decomposition delegation fidelity. The harness also records routing macro-F1, per-class routing metrics, success rate, latency, cost, and orchestration efficiency for larger suite-level comparisons.

4 EXPERIMENTAL SETUP

Reported confidence intervals use 95% bootstrap CIs; paired bootstrap significance tests (α = 0.05) are used where paired comparisons are defined.

Table 2: Routing policies under comparison.

Policy	Type	What it tests
Fixed	Baseline	Zero-intelligence lower bound (always one route)
Heuristic	Baseline	Rule/keyword routing on complexity + flags
LLM-as-Router	Test	Model decides routing per task by reasoning over description + metadata
Retry(heuristic)	Test	Adds automatic retry — does retry alone contain cascades?
Oracle	Ceiling	Routes to the gold label (upper bound)

Table 3: Experiment 1: routing accuracy by case type.

Policy	Overall	Aligned	Adversarial
Fixed	23%	25%	20%
Heuristic (keyword/flags)	62%	**100%**	**0%**
LLM-as-Router (Sonnet 4.6)	**100%**	100%	**100%**
Oracle (ceiling)	100%	100%	100%

5 RESULTS

5.1 Experiment 1 — Routing Policy Comparison (measured)

We isolate the routing-mechanism question on a focused diagnostic of **26 expert-labelled gold cases** covering all four routing decisions: 16 *aligned* (surface flags match intent) and 10 *adversarial* (flags missing or misleading, but the description's intent is unambiguous). The model-driven router is Claude Sonnet 4.6 via forced structured (tool-use) output; results are stable across 3 independent passes (78/78), and the prompt never sees the gold label.

Finding. The heuristic is perfect on aligned cases but collapses to **0%** on adversarial ones; the model-driven router recovers **all** of them (0% → 100%), matching the oracle on this diagnostic. Because the same tasks are solved by an intent-reasoning router, the heuristic's adversarial failure is best explained by the routing **mechanism** (keyword/flag matching vs. reasoning over intent), not by inherent task difficulty. Larger workflow-suite evaluation is the intended setting for a graded comparison once the diagnostic no longer saturates.

5.2 Experiment 2 — Failure Injection and Recovery (real Claude measured run; core)

Design. Five MAST failure modes — *ambiguous delegation*, *tool-invocation error*, *context pollution*, *conflicting sub-agent outputs*, *premature action* — injected at the **prompt level** into a verifiable arithmetic dependency chain executed by a **real Claude agent** (Sonnet 4.6), under fixed, heuristic, and retry policies. Real recovery / cascade are measured by exact-match against ground truth; the measured-run module is included in the repository.

Real measured findings (Sonnet 4.6, N=30). The tool-invocation mode is **fully recovered** (recovery 1.0, cascade radius 0) — the agent computes by hand when the tool fails. The four **latent/semantic modes** all **fail** (final-task success **0.0**) and **cascade to every downstream stage** (cascade radius 2 of 2). Critically, the retry policy **does not recover them**: retrying while the failure is still present reproduces the error and only **lengthens time-to-detection**. This real-agent result supports the mechanism: retry repairs retryable tool faults but not failures needing attribution, state repair, or semantic validation (RQ3/RQ4). Bootstrap CIs are degenerate here (latent modes 0.0 [0.0, 0.0]; tool fault 1.0 [1.0, 1.0], n=6 each), reflecting the controlled construct and small per-mode sample, not statistical strength (§7).

Table 4: Experiment 2: per-mode final-task success and cascade radius, arithmetic chain vs. loan-approval domain (real Claude, N=30 each; 95% bootstrap CI).

	Arithmetic		Domain	
Failure mode	**Success**	**Cascade**	**Success**	**Cascade**
tool_invocation_error	1.00	0.00	0.67	0.67
ambiguous_delegation	0.00	2.00	0.50	1.00
context_pollution	0.00	2.00	0.00	2.00
conflicting_outputs	0.00	2.00	0.00	2.00
premature_action	0.00	2.00	0.00	2.00

Table 5: Experiment 3: mean cascade radius by pipeline depth (real Claude, N=90; latent modes pooled, n=24/depth; 95% bootstrap CI). Linear in depth: cascade ≈ depth − 2.

Depth	Latent cascade radius	Tool cascade radius
3	1.00 [1.00, 1.00]	0.00
5	2.88 [2.63, 3.00]	0.00
7	5.00 [5.00, 5.00]	0.00

Domain-grounded validation (loan-approval workflow, N=30). To test whether these signatures are an artifact of the abstract arithmetic chain, we re-ran the *identical* failure injection on a domain-grounded variant: the same verifiable computation reframed as a multi-role loan-approval pipeline (Intake Officer → Risk Analyst → Compliance Officer → Approval Manager), each stage prompted in business terms. **The core ordering mostly holds**: context pollution, conflicting outputs, and premature action still fail completely, while tool faults remain the most recoverable (Figure 1). Crucially, the **absolute rates shift with framing**: ambiguous delegation rises to **0.50** (the business-role context lets the agent infer the intended operation about half the time) and tool-invocation recovery falls to **0.67**. This framing-sensitivity is evidence of model behavior under context, not a purely structural tautology, and directly addresses the construct-validity concern (§7).

5.3 Experiment 3 — Cascade Propagation Depth (real Claude measured run, N=90; core)

Design. Inject a single seeded error at stage 1 of variable-depth (3-, 5-, 7-stage) pipelines and measure how far it propagates. **Cascade radius** (downstream stages corrupted) is a central differentiator vs. MAS-FIRE, which has no stages-traversed metric.

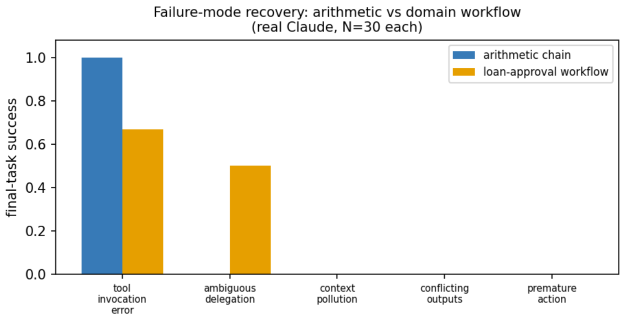

Figure 1: Experiment 2 — failure-mode final-task success, arithmetic chain vs. loan-approval workflow; the failure-mode *ordering* is robust across framings while absolute rates shift (real Claude, N=30 each).

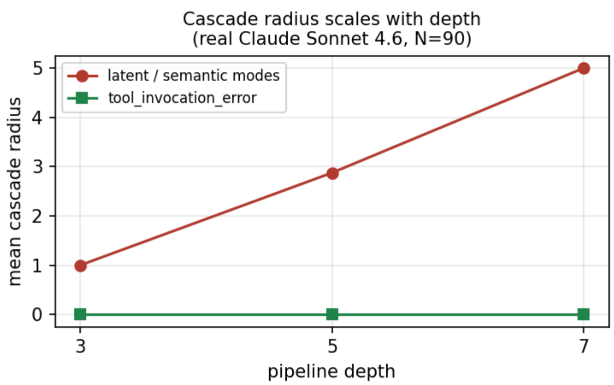

Figure 2: Experiment 3 — mean cascade radius vs. pipeline depth; latent/semantic modes scale 1.0 → 2.9 → 5.0 across depths 3/5/7 while tool faults stay at 0 (real Claude, N=90).

Real measured findings (Sonnet 4.6, N=90). Across the four latent/semantic failure modes, **mean cascade radius scales monotonically with depth: 1.0 → 2.9 → 5.0 at depths 3 / 5 / 7** (Table 5, Figure 2) — the failure propagates to essentially every downstream stage (depth − 2). Tool-invocation errors stay at cascade radius 0 at every depth (the agent recomputes by hand). The lone deviation is ambiguous delegation at depth 5 (mean 2.5 vs. 3.0 for the other latent modes), where the real agent occasionally guesses the intended operation correctly — a realistic-noise signature absent from a deterministic model. **This is the paper's main cascade-depth result and a primary differentiator vs. MAS-FIRE.**

5.4 Experiment 4 — Decomposition Quality (real Claude measured run, N=20; secondary)

For composite tasks $(a\, op\, b)\ op\ (c\, op\, d)$ with a canonical 3-step gold decomposition, we score the produced decomposition against gold — **delegation fidelity** (gold sub-results recovered), **granularity error**, **wasted sub-tasks** — comparing a `monolithic` (single-step) vs. a `decompose` (planner) policy.

Table 6: Experiment 4: decomposition quality, decompose vs. monolithic (real Claude, N=20; 95% bootstrap CI). Paired bootstrap over the 10 shared tasks.

Policy	Deleg. fidelity	Granularity err.	Final correct
decompose	1.00 [1.00, 1.00]	0.00 [0.00, 0.00]	1.00
monolithic	0.37 [0.33, 0.43]	2.00 [2.00, 2.00]	1.00
paired Δ	fidelity +0.63 [0.57, 0.67], $p < 0.0001$; granularity −2.0, $p < 0.0001$		

Table 7: Policy-conditioned containment on latent failures (real Claude, N=75; 95% bootstrap CI).

Policy	Latent recovery	Cascade radius
fixed / heuristic / retry	0.08 [0.00, 0.25]	1.83 [1.50, 2.00]
LLM-as-router	**0.83 [0.58, 1.00]**	**0.33 [0.00, 0.83]**
Oracle (ceiling)	1.00 [1.00, 1.00]	0.00 [0.00, 0.00]

Real measured findings (Sonnet 4.6, N=20). The `decompose` policy produces a faithful three-step decomposition (delegation fidelity **1.00**, granularity error **0**), while `monolithic` reaches the correct final answer but exposes no delegable sub-structure (fidelity **0.37 [0.33, 0.43]**, granularity error **2**). This fidelity gap is large and statistically significant under a paired bootstrap over the 10 shared tasks: +0.63, **95% CI [0.57, 0.67]**, $p < 0.0001$ (Table 6). Notably, `final_correct` is **1.0 for both** — the arithmetic is easy enough that both get the answer; the discriminator is the **decomposition structure**, not final correctness. For multi-agent orchestration this is exactly the point: a monolithic agent can be *right* yet produce nothing a planner can delegate, audit, or recover from.

5.5 Policy-conditioned containment probe (real Claude, N=75 + N=225)

Experiments 2–3 pool over baseline policies; we add **LLM-as-router** and **Oracle** policies to ask whether the routing policy itself can govern containment. **The LLM-policy semantics are an explicit modelling assumption**: at the injected stage the router is given the trusted upstream value and licensed to detect and correct the anomaly. We therefore treat the LLM row as a strong self-correction probe rather than a clean estimate of deployable routing without trusted state.

Policy probe by depth (N=225). Under this stronger trusted-state semantics, the policy effect *amplifies with pipeline depth*: baseline cascade radius grows with depth (0.9 / 2.7 / 4.6 at depths 3 / 5 / 7), while the LLM router stays nearly flat (0.25 / 0.75 / 0.83) and Oracle fully contains (0 / 0 / 0). At depth 7 the LLM router cuts cascade radius ~**5.5**× versus the baselines. This suggests that containment benefits can become larger in deeper pipelines, while the deployable-routing version remains future work (Figure 3). The policy CSVs are committed under `data/measured/`.

Ablation: the model, or the hint? The rows above hand the router the trusted upstream value — a possible information leak. Re-running Exp 2 with an `llm_noupstream` policy (same self-correction,

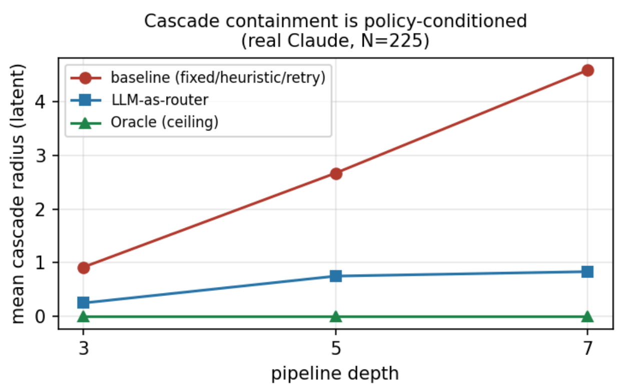

Figure 3: Experiment 3 — mean latent cascade radius vs. pipeline depth per routing policy; baseline grows with depth while the trusted-state LLM probe stays nearly flat and Oracle fully contains (real Claude, N=225).

no trusted-upstream hint) collapses latent recovery from **0.75 [0.50, 1.00]** to **0.17 [0.00, 0.42]**, barely above the baseline 0.08 (paired drop **0.58 [0.17, 0.92]**, $p < 0.01$). The LLM policy's containment is thus *mostly the trusted-upstream signal, not autonomous detection*: we read the LLM column as a trusted-state probe, not evidence that an LLM router autonomously contains latent cascades (`exp2_ablation_real.csv`).

6 DISCUSSION

Experiment 1 establishes the paper's first reproducible claim: routing reliability depends on *mechanism*. Experiments 2 and 3 deliver the main reliability result on a controlled verifiable chain: the **failure-mode gap** is measured (four latent modes at 0.0 final-task success vs. a fully-recovered tool fault), and **cascade radius scales with pipeline depth** (1.0 → 2.9 → 5.0 at depths 3 / 5 / 7) — a stages-traversed signature not emphasized by prior reliability benchmarks. Critically, **retry does not eliminate latent cascades**: it reproduces the failure and lengthens time-to-detection, so *detection/attribution* — not blind retry — is the necessary containment mechanism. The policy-conditioned extension shows the potential value of trusted state repair, but its deployable interpretation is deliberately limited (§7). We are deliberate that these are mechanism probes on a controlled chain; domain-workflow validation is the next step.

7 LIMITATIONS

- The Exp 1 diagnostic is small (26 cases) and *saturates* for a capable model — it is a mechanism probe, not a difficulty benchmark; the larger workflow-suite results are needed for a graded comparison.
- Workflows are synthetically generated; real-world pipeline validation is future work.
- Single model family reported for the LLM router; a cross-model sweep is future work.
- **Construct validity of Exp 2/3.** The failure-recovery and cascade experiments run on a *verifiable arithmetic dependency chain*, chosen for clean ground truth, not a domain workflow. In this construct cascade radius is **partly by design** — a downstream stage consumes the upstream value, so a latent corruption propagates to ~(depth − 2) stages — so we present Exp 2/3 as **controlled mechanism probes**. The evidence that this measures model behavior rather than a tautology is the non-determinism we observe (ambiguous delegation at depth 5 yields mean cascade 2.88, below the structural maximum) and the domain-grounded re-run (Figure 1): the failure-mode *ordering* mostly persists across framings while absolute rates shift with context. Full finance / HR / DevOps suite validation remains future work.
- **Single LLM, not a literal multi-agent system.** Exp 2/3 execute one Claude agent over a staged chain with prompt-level fault injection — simulating orchestration failure modes rather than wiring N independent agents; a real multi-agent harness is future work.
- **LLM-policy semantics are a modelling assumption.** The policy-conditioned results define the LLM router as a stronger pass given the trusted upstream value and licensed to self-correct; the absolute LLM numbers depend on this choice. The Oracle (gold-repair ceiling) and baseline columns are unambiguous, but the LLM column should be read as a trusted-state self-correction probe, not yet as a deployable routing estimate. To isolate whether the LLM gain reflects the model detecting the fault versus being handed the trusted upstream, we ran an ablation policy (`llm_noupstream`) that drops the trusted-upstream hint: latent recovery collapses to near-baseline (§5.5), confirming the LLM column is a trusted-state probe, not autonomous detection.

8 ETHICS AND BROADER IMPACT

OrchestraBench evaluates the *reliability* of automated orchestration. Improving failure attribution and cascade containment is intended to make production AI systems **safer, more auditable, and cheaper to operate** — surfacing silent failures before deployment rather than after.

- *Leaderboard overfitting*: a public reliability benchmark can incentivize tuning to the injected failure modes. Mitigation: seed-controlled, **held-out** failure scenarios, and reporting cost alongside accuracy where deployment-cost comparisons are made.
- *Dual use*: cataloguing how orchestrators fail could inform adversarial prompting. Mitigation: the injected modes are already public (MAST); we add measurement, not new attack surface.
- *Over-trust*: a high score must not be read as production safety. We state scope limits (synthetic workflows; mechanism probes) explicitly.

AI-use disclosure. Generative AI tools were used for limited editorial and coding assistance; all results, citations, and claims were checked against committed code, data, and cited records.

No human-subjects data is used; all workflows are synthetic and include no real applicants, employees, customers, or proprietary records.

9 REPRODUCIBILITY

Failure scenarios are regenerable from fixed seeds, and every reported aggregate is recomputable from committed artifacts. The Exp 1 reproduction script regenerates the §5.1 table from the committed gold set (offline baselines need no key; the LLM row runs when the Anthropic API key is set). The measured analysis script recomputes every Exp 2/3/4 number above (per-mode recovery, cascade radius by depth, decomposition fidelity) with bootstrap 95% CIs and paired significance tests, directly from the committed CSVs (offline, no key). Code is Apache-2.0; data and failure scenarios are seeded; the full test suite (151 tests) runs offline on Python 3.10/3.11/3.12. Total real-LLM cost across the project is ≈ \$1.50 (Sonnet 4.6; short arithmetic prompts), so reproduction is negligible.

10 CONCLUSION

OrchestraBench reframes multi-agent evaluation from "did the task succeed?" to "did the orchestrator route, recover, and decompose reliably?" Experiment 1 delivers a clean, reproducible result — model-driven routing closes a 0%→100% adversarial gap that keyword routing cannot — while the failure-injection and cascade experiments measure, on a controlled chain, the reliability axis production teams most need: latent failures that retry cannot repair, and a cascade radius that grows with pipeline depth (1.0 → 2.9 → 5.0 across depths 3 / 5 / 7).

REFERENCES

[1] Anonymous (concurrent work). 2026. IntentBench: Intent Specification as a First-Class Evaluation Object. Concurrent spec-layer benchmark; complementary, cross-cite.

[2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657 [cs.AI] Introduces the MAST taxonomy: 14 failure modes in 3 categories; MAST-Data 1,600+ traces over 7 frameworks; Cohen's kappa = 0.88.

[3] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. arXiv:2604.22708 220 annotated failure traces; full-execution-observability attribution.

[4] Jin Jia, Zhiling Deng, Zhuangbin Chen, Yingqi Wang, and Zibin Zheng. 2026. MAS-FIRE: Fault Injection and Reliability Evaluation for LLM-Based Multi-Agent Systems. arXiv:2602.19843 15 fault types (8 intra-/7 inter-agent); injection via prompt / response / message-routing; tested on MetaGPT, Table-Critic, CAMEL.

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR).* arXiv:2310.06770; 2,294 real GitHub issues over 12 Python repos.

[6] Siddhant Kulkarni and Yukta Kulkarni. 2026. Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies. arXiv:2603.22651 10k SEC filings; reflexive F1 0.943 @2.3x, hierarchical 0.921 @1.4x.

[7] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR).* arXiv:2308.03688; 8 environments, 27 LLMs.

[8] Aaron Xuxiang Tian, Ruofan Zhang, Jiayao Tang, Young Min Cho, Xueqian Li, Qiang Yi, Ji Wang, Zhunping Zhang, Danrui Qi, Zekun Li, Xingyu Xiang, Sharath Chandra Guntuku, Lyle Ungar, Tianyu Shi, and Chi Wang. 2025. Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks. arXiv:2509.23537 GPQA-Diamond: at least one agent correct in 95.5% of cases vs. 87.4% orchestrated.

[9] Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025. OdysseyBench: Evaluating LLM Agents on Long-Horizon Complex Office Application Workflows. arXiv:2508.09124 602 long-horizon office tasks (300 real + 302 synthesized).

[10] Yizhe Xie, Congcong Zhu, Xinyue Zhang, Tianqing Zhu, Dayong Ye, Minfeng Qi, Huajie Chen, and Wanlei Zhou. 2026. From Spark to Fire: Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration. arXiv:2603.04474 Error cascades in agent pipelines.

[11] Geunbin Yu. 2026. AdaptOrch: Task-Adaptive Multi-Agent Orchestration in the Era of LLM Performance Convergence. arXiv:2602.16873 Task-adaptive orchestration method (+12–23%).

[12] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML), Spotlight.* Who&When dataset: failure logs from 127 multi-agent systems. arXiv:2505.00212.